

Module 9 Assignment: Generative Artificial Intelligence

JHU EP 605.206 - Introduction to Programming Using Python

Introduction

Retrieval-Augmented Generation (RAG) models, utilize foundation models to generate contextually accurate results by referencing information in a provided knowledge base.

In this Assignment, you will build a RAG solution which retrieves information from NASA's APOD (Photo of the day) API for a specified date. Based on the image description retrieved, you'll obtain more information in the form of Wikipedia summaries. The LLM will then write a more comprehensive summary of what the photo of the day contains using both the originally provided information and the Wikipedia summaries.

Skills: conditional statements, control structures, file I/O, functions, classes, REST API's, Python requests

Large Language Models

A **Large Language Model (LLM)** is a type of artificial intelligence designed to understand and generate human-like text. These models are trained on vast amounts of written content—everything from books and articles to code and social media—to learn how language works. Popular examples include models like GPT, Claude, and LLaMA. They're built using something called the transformer architecture, which allows them to consider the relationships between words in a sentence or paragraph, making their responses more coherent and context-aware.

LLMs work by predicting what word (or token) should come next in a sequence of text. For example, if you start a sentence, the model will try to guess the next most likely word based on everything you've typed so far. This is made possible by a mechanism called "attention," which helps the model focus on the most relevant parts of a sentence when making predictions. During training, the model adjusts billions of internal settings—called parameters—to improve its accuracy.

Building an LLM takes a lot of resources. First, it needs a huge and diverse dataset—typically gathered from sources like Wikipedia, open-access books, websites, and forums. Then, massive computing power is required, often involving thousands of powerful GPUs or CPUs running for weeks or even months. The model is carefully designed using deep learning techniques, often relying on transformer-based architectures, and trained using objectives like predicting the next word or filling in missing words.

Despite their impressive abilities, LLMs aren't perfect and can fail in several ways. They sometimes generate information that sounds correct but is completely made up—this is called a hallucination. They can also reflect biases present in their training data, struggle with complex reasoning, or lose track of context in longer conversations. Additionally, unless they're connected to recent data sources, their knowledge can become outdated over time.

To improve their performance on specific tasks or attempt to provide more accurate information, developers often use techniques like fine-tuning or **retrieval-augmented generation (RAG)**. Fine-tuning involves re-training the model on a smaller, specialized dataset so it performs better in a specific area, like medical or legal language. A RAG, on the other hand, connects the model to an external database or search engine. This allows it to "look up" relevant facts in real time, improving the accuracy at answering questions about current or detailed topics without needing to retrain the entire model.

Retrieval Augmented Generation

Retrieval-Augmented Generation, or **RAG**, is a powerful technique that enhances how language models (LLMs) answer questions. You can think of RAG as a sort of wrapper for an LLM - enhancing its capabilities by integrating external knowledge sources for more accurate and relevant responses. *It is very important to note that using an LLM in such a way does not undo or overwrite the language relationships which have been already trained into the model. Both RAGs and fine-tuning do NOT eliminate hallucinations, instead, they are an effort to reduce them.*

In simple terms, RAG combines search with generation. Instead of relying only on what the model learned during its original training, it can now search a connected knowledge base to find relevant information. This is the **"Retrieval"** step.

Next, the **"Augmentation"** happens. The content that was retrieved from the knowledge base is added to the prompt sent to the LLM. This means the model is not just relying on its internal memory but is also using up-to-date, accurate information from a trusted source. Finally, the **"Generation"** step occurs. The LLM uses natural language processing (NLP) to create a clear and coherent response based on the combined prompt.

So why is RAG useful? First, it allows you to use a trusted source of information—like company documents, academic papers, or internal wikis—while still getting the natural, readable responses LLMs are good at producing. It gives you flexibility to work with your own data instead of relying solely on what's built into the model. It's also cost-effective because it eliminates the need for expensive and time-consuming fine-tuning of the model. One of the biggest advantages is that RAG can provide up-to-date information, since it pulls from live or recently updated sources. This also helps reduce mistakes by grounding the model's answers in real data.

A basic flow of a RAG solution looks like this:

1. **Input** – You provide a question or prompt to the system.
2. **Knowledge Base** – Relevant documents are retrieved from a knowledge source.
3. **LLM** – The language model takes the original question plus the retrieved information.
4. **Enhanced Prompt** – This combined data forms an enhanced prompt that gives the model extra context.
5. **Results** – The LLM then generates a final, detailed, and well-informed answer.

This process helps deliver smarter, more relevant answers using a combination of AI language skills and real-time knowledge.

Build your RAG Pipeline

The purpose of the RAG solution you're building is to retrieve NASA's Photo of the Day for a specific date and enhance the explanation of the photo to give a more detailed and accessible description. Retrieving the photo of the day from NASA's APOD API returns information along with a url to the image location. Within that information, there is an 'explanation' of the image. The provided explanation is not always very complete. Your RAG solution will take that brief explanation and expand upon that.

After retrieving the information for the photo of the day (for a specified date), the first step within the RAG will be to have the LLM interpret the photo explanation and return a list of terms which it deems important based on the explanation. To accomplish this, you'll send the photo title and explanation (found in the results from the API call to NASA's APOD) along with a carefully worded prompt which we've called **lookup_prompt** (provided below). The LLM will take the provided photo explanation with the **lookup_prompt** and **return a single string** written in the form of a list of strings where each string is the term you'll be retrieving information from Wikipedia on. *Note: While it appears to be a list of strings, you'll see that it is actually a single string returned. You'll need to convert it into a true list of strings to use.*

Now that you have your list containing terms, you'll gather more information on each of those terms by using the Wikipedia library (details below) to retrieve a summary. You'll join the summary results into one long string (separated by the search terms used). This will **act as your knowledge base** along with the original explanation from the photo of the day API call.

You now have all of the parts your RAG solution needs. You'll send the original explanation from the photo of the day API call with your Wikipedia summary (your knowledge base) and a carefully worded prompt called **rag_prompt** (provided below) to the LLM. The LLM will generate a new description of the photo of the day utilizing the knowledge base we have provided and the **photo_explanation**.

We will break up the assignment into five python files (main.py, rag.py, apod.py, wiki.py, api.py) each of which is described below in the requirements section. We have provided part of wiki.py to you. You will need to complete the code. There are instructions below on how to do this as well as comments within the code base to help you complete it.

Note: DO NOT modify any lines of code in wiki.py that we provide to you. There are elements of Python we utilize in the provided code which are considered OOS for you, which is why we have provided them to you.

Prompts

In this assignment, the prompt refers to the input that is sent to the language model (LLM) to guide its response. Prompts can include built-in instructions (pre-written context or rules) as well as user-generated questions that the model needs to answer. It's important to be careful with the wording of prompts because the language model interprets everything very literally. Clear, precise wording helps the model generate more accurate and relevant answers. Poorly phrased prompts can lead to confusing, incomplete, or off-topic responses. Don't worry, we provide you with carefully worded prompts to use.

There are two prompts this RAG solution will use:

1. The first, **lookup_prompt**, will be used to determine what terms should be looked up in Wikipedia articles to get more information for the updated photo description that the LLM will generate. In addition to the prompt wording shown below, you will also provide the **explanation** from the API call result as a string. In the prompt below, we referred to this variable as **photo_explanation**, though you may call it whatever you like.
2. The second prompt, **rag_prompt**, will be provided to the LLM along with the original **photo_explanation** (from the APOD API retrieval results) and the knowledge base (summaries from the Wikipedia entries) to generate the new photo explanation.

lookup_prompt:

*"Analyze the following and based on the information provided, create a list of terms to look-up for more information. The list should be returned as a list of strings in python format. No other information should be returned. Information: " + **photo_explanation** (as a string).*

rag_prompt: (Note: terms in brackets are variable information)

*""NASA APOD Title: [**photo_title**]*

*Explanation: [**photo_explanation**]*

Additional Context from Wikipedia:

[summary of Wikipedia content]

Write a more detailed and accessible description of the image using all of the above.

""

Additional Information

Outside of an academic environment, writing production code often involves research and reading into using new Python libraries or unfamiliar APIs. Reading documentation is a large part of new code development. We've given you some examples of this as the semester has progressed though now, you'll be doing a bit more of that for this assignment. **You'll find the following very helpful in successfully completing this assignment.**

- NASA's APOD API documentation can be found [here](#). You'll need to [create a free account to obtain your own API key](#) and you'll use the APOD url: <https://api.nasa.gov/planetary/apod>.
- For the reference information, you'll be using a Wikipedia wrapper written by someone else. Their wrapper has been added to the Python Packaging Index library, [PyPI](#), which makes it easy to install and use like a normal python library. Information for the wikipedia 1.4.0 library and instructions for use can be found [here](#).
 - *Note: We provide part of wiki.py to you. You will still need to review the package documentation to complete the code.*
- The LLM we will use is Ollama, though rather than create API calls directly to the LLM, you'll be using the [Ollama Python Library](#). You'll need to familiarize yourself with the documentation regarding the prerequisites, installation, and usage. We will **use the model 'llama3.2'**. There are some tips in the next section on getting Ollama properly working on your computer.

Installing and Running Ollama Locally

In order to use the Python Ollama library, you'll need to do the following:

1. [Download](#) and install Ollama to your computer.
2. Make sure Ollama is running. (You'll double click on the installed icon to do this).
3. Open a terminal window.
 - a. Verify ollama is installed and running properly by typing '**ollama**' into the command line. Some basic instructions and options for ollama should print to your terminal.
 - b. Pull the specific LLM model we are using (llama3.2) for this assignment using the command: '**ollama pull llama3.2**' *Note: Installing Ollama from part 1 does not specify the model we want nor install our model selection. That's what this step does.*
 - c. Verify you have correctly pulled the model by typing: '**ollama list**'. You should see the model you just pulled (llama3.2) listed. It will look something similar to this:

```
(base) vanessalynch@vanessas-mbp-3 gen_ai_assignment % ollama list
NAME                ID              SIZE    MODIFIED
llama3.2:latest      a80c4f17acd5    2.0 GB  2 months ago
```

4. Install the [python package ollama](#) as you normally would. Example: '**pip install ollama**'. *Note: this installs the python library so that python can now access the version of ollama you installed locally in steps 1 – 3 above.*

Assignment Requirements

1. **Your solution must use the provided prompts EXACTLY as provided.** RAG solutions are extremely sensitive to the wording used in the prompts and the results of your RAG solution can vary wildly with minor changes in word choice. **Do NOT change or alter or add to this in any way.**
2. Your solution **must** make use of classes, the main method, requests, datetime, conditionals, [Ollama's Python Library](#) (model = llama3.2), [wikipedia 1.4.0](#) from PyPI, [NASA's APOD API](#), and any further specifications outlined below.
3. Create a file called **main.py** that imports any necessary modules and uses `__name__` to call a function **run_rag()**. The **run_rag()** function should:
 - a. Take an argument 'date' which is a string in the format "XXXX-XX-XX" (year-month-day). The date which will be provided in your main method will be: "2024-12-14". *Note: The run_rag() function should use the current date by default if no date is provided. Use [datetime](#) to obtain the current date.*
 - b. Instantiate the APOD class (from apod.py) using your NASA API key and use its `fetch_apod(date)` method to retrieve the APOD information for the date provided in 3.a (above). Prints "Original APOD Image Information:" followed by the explanation result to the terminal and to the output file named results.txt in the format shown in the sample output.
 - c. Initialize your RAG class (from **rag.py**) and generate your lookup list with the **generate_lookup_list()** method.
 - d. For each of the terms in the resulting lookup list, retrieve the Wikipedia summary information using the function **fetch_wikipedia_summary()** from **wiki.py**. Add all of the retrieved summaries to a single string which is separated by whitespace and contains the search term before the associated information.
 - e. Send the results of the (3.b) and (3.d) to the **generate_augmented_description()** method of the RAG class. Prints "RAG Generated Description:" followed by the returned result to the terminal and to the output file named results.txt in the format shown in the sample output. *Note: No additional formatting of the LLM generated output is required. You can print that exactly as it is returned to you.*
 - f. Finally, retrieve the APOD photo.
 - i. Prints "Link to this photo:" to the output file named **results.txt**.
 - ii. If an HDURL is available in the result from 3.b., print that link to **results.txt**. Otherwise, print the link to the URL (returned in 3.b.) to **results.txt**.
 - iii. Based on if the HDURL or URL is available, save the image as **"nasa_apod_image.jpg"** using the **download_image()** method from the APOD class.
4. Next, create a **RAG** class inside of a module named **rag.py**. Your class should:
 - a. Import chat from ollama. *Note: You may find this documentation on the [Python Ollama Library](#) helpful in the creation of this class.*

- b. Contains a constructor that takes as an argument the LLM model to use ('llama3.2') and defines the model as an instance variable.
 - c. Contains a method named **rag_response** which takes a prompt (string) as an argument. You'll use your import chat (from ollama) within this method to send the relevant information to the LLM for analysis. Return the content of the LLM's message.
 - d. Contains a method named **generate_lookup_list** which takes the APOD API retrieved information and a file name as an argument. This method:
 - i. Contains the **lookup_prompt**.
 - ii. Calls the **rag_response** method (with the **lookup_prompt**).
 - iii. Converts the resulting single string from (ii) to a list of strings where each string is a search term(s).
 - iv. Prints the following to the file **results.txt**:
 1. "Additional Search Terms:"
 2. For each term in the list of terms, print a bullet point (using Unicode) followed by the term. Each new bullet + term is printed on a new line. See the Sample output from results.txt for an example. *Note: Any bullet point can be chosen for this. The requirement is that the bullet point be represented in the method using Unicode.* You may find the [Python Unicode documentation](#) and this [Unicode table](#) helpful.
 - v. Returns the list of terms
 - e. Contains a method named **generate_augmented_description** which takes the APOD API retrieved information and the results from the joined Wikipedia summary results as arguments. This method:
 - i. Contains the **rag_prompt**.
 - ii. Calls the **rag_response** method (with the **rag_prompt**).
 - iii. Returns the result
- 5. Now, complete the module named **wiki.py** which we have provided on Canvas. This module will import the wikipedia library and contain a function named **fetch_wikipedia_summary()**. This function should:
 - a. Take as input a single search term and a file name as arguments.
 - b. Replace any spaces within the search term argument with a single underscore ('_').
 - c. Call the search method within the [wikipedia library](#) and search for the term from 5.b.
 - d. If no results are returned from 5.c, print to **results.txt**:
 - i. "No articles found for '[term]'..."
 - e. If there are results from 5.c are returned, then:
 - i. Take the results and if more than one result is returned, use just the first one. (Ignore the rest).
 - ii. Retrieve a summary from the wikipedia method named summary using the first returned result.
 - iii. To **results.txt** print:
 1. "Searching Wikipedia for '[term]'..."
 2. "Summary of '[wikipedia entry result]':" followed by the summary returned from 5.e.i.

- f. If steps 5.a – 5.e result in an error (handled in our provided `wiki.py` by 'except') then:
 - i. To **results.txt**, print "An issue occurred retrieving information for [term]."

DONE 6. Next, create an **API** class inside of a module named **api.py**. Your python file should import requests and your class should contain:

- a. A constructor which:
 - i. Takes in a base URL and API key as parameters. If no API key is provided, a default value of None should be set.
- b. A method named **get_response()** which:
 - i. Takes in parameters for an API request.
 - ii. Sends a GET request to the API using the stored base URL and the provided query parameters. Returns a JSON object from the API request.

7. Lastly, you'll create a module named **apod.py** that defines an **APOD** class extending the API (parent) class from `api.py`. This class should utilize the requests library and implement:

- a. A constructor which:
 - i. Takes in the API Key as a parameter (Your unique NASA API Key will be passed in here when the class is instantiated).
 - ii. Calls the parent API class constructor and initialize it with the APOD URL (provided above) and API Key.
- b. A method named **fetch_apod()** which:
 - i. Takes a date in string format (outlined previously) as an argument.
 - ii. Gets the [API response for the provided date](#).
 - iii. Returns the response in JSON format.
- c. A method named **download_image()** which:
 - i. Takes a URL and filename as arguments.
 - ii. Uses `requests.get` with image URL to download the image.
 - iii. If the `status_code` of the response returned equals 200, then:
 - 1. Write the result (in binary) of 6.d.ii to the filename passed in. *Note: This is not results.txt. This file should be named 'nasa_apod_image.jpg'.*
 - 2. To the terminal/screen print: "Image successfully saved as " then list the filename.
 - iv. Otherwise to the terminal/screen print: "Failed to download image. Status code: " followed by the value of the status code.

Restrictions

As a reminder, all out-of-scope concepts as outlined in the Syllabus are not permitted. If you'd like to make use of additional Python features outside of the Reading and Video Lectures, please send us a note on Teams so we can weigh-in with a decision.

Concepts that are **not permitted** for use on this Assignment include, but are not limited to, the following:

- `enumerate()`, `map()`, `join()`, `zip()`, `pop()`, `sorted()`, `format()`, `extend()`, `isinstance()`, `typing()`, `deque()`
- List/dictionary/set comprehensions
- Generator expressions
- Lambda functions
- Exceptions, `try/except`, `try/finally`, `raise`, `ValueError` (and all other error types)
- Dummy variables, global variables, `global`
- Any imported libraries (with the exception of those outlined in the assignment or provided in starter code).
- Argument unpacking
- bytes I/O
- hexadecimal literals
- `os`, `pathlib`
- `re`, `eval`, `ast`, `ast.literal_eval`

If you'd like to make use of additional Python features outside of the Reading and Video Lectures, please send us a note on Teams so we can weigh-in with a decision.

Sample Output

Note: Due to the nature of an LLM, the output results will differ slightly as well as the FORMATTING which the LLM returns the information in. You DO NOT need to modify any formatting returned by the LLM. Print the information the LLM returns as it is given to you – adding the section headers and some white space between sections (roughly as shown below).

Example Terminal/Screen Output:

```
(base) vanessalynch@vanessas-mbp-3 gen_ai % python3.10 main.py
Original APOD Image Information:

Awkward and angular looking, Apollo 17's lunar module Challenger was designed for flight in the near vacuum of space. Digitally enhanced and reprocessed, this picture taken from Apollo 17's command module America shows Challenger's ascent stage in lunar orbit. Small reaction control thrusters are at the sides of the module with the bell of the ascent rocket engine underneath. The hatch allowing access to the lunar surface is seen at the front, with a round radar antenna at the top. Mission commander Gene Cernan is clearly visible through the triangular window. This spaceship performed gracefully, landing on the Moon and returning the Apollo astronauts to the orbiting command module in December of 1972. So where is Challenger now? Its descent stage remains at the Apollo 17 landing site in the Taurus-Littrow valley. The ascent stage pictured was intentionally crashed nearby after being jettisoned from the command module prior to the astronauts' return to planet Earth.

RAG Generated Description:

This photograph showcases a historic moment in space exploration, taken during the Apollo 17 mission in December 1972. The image captures the lunar module Eagle, piloted by Eugene Cernan and Harrison Schmitt, as it descends onto the surface of the Moon's south pole.

In the foreground, the landscape stretches out into the vast expanse of Taurus-Littrow valley, a region located on the near side of the Moon at coordinates 20.0°N 31.0°E. The valley was chosen by the Apollo 17 crew as their landing site due to its unique geological features and scientific significance.

The lunar module's descent stage can be seen in the distance, while the ascent engine, a fixed-thrust rocket engine developed by Bell Aerosystems, is visible on the side of the module. This engine was used to propel the spacecraft back into lunar orbit after the landing.

Reaction control thrusters, small engines that provide attitude control and translation, are also visible on the exterior of the lunar module. These systems played a crucial role in stabilizing the spacecraft during its descent and ascent.

The reaction control thrusters are paired with the ascent rocket engine, which is a hypergolic rocket engine that uses Aerozine 50 fuel and N2O4 oxidizer to produce thrust. This engine was used to propel the lunar module away from the Moon's surface and back into orbit.

As the spacecraft takes off, it can be seen leaving behind a cloud of debris, including regolith, the fine dust that covers the Moon's surface. The lunar module is also accompanied by two astronauts, Cernan and Schmitt, who are suited up in their spacesuits to protect themselves from the harsh conditions on the Moon.

The image also includes several notable features of the Taurus-Littrow valley, including the Littrow crater, which can be seen in the distance. The valley is composed primarily of feldspar-rich breccia, a type of rock that forms when fragments of other rocks are welded together by heat and pressure.

Overall, this photograph captures an iconic moment in space exploration history, showcasing the technological achievements of the Apollo 17 mission and providing a glimpse into the fascinating geology and landscape of the Moon's surface.

Image successfully saved as 'nasa_apod_image.jpg'
```

nasa_apod_image.jpg



Example of results.txt:

Note: The line numbers are displayed by the IDE and are NOT part of the results.txt output you will write. Also, the url at the very end of this file is interpreted as a hyperlink (underlined) by the IDE. You do NOT need to underline the link. Print the url as a string without any formatting.

```
1 Original APOD Image Information:
2
3 Awkward and angular looking, Apollo 17's lunar module Challenger was designed
  for flight in the near vacuum of space. Digitally enhanced and reprocessed,
  this picture taken from Apollo 17's command module America shows Challenger's
  ascent stage in lunar orbit. Small reaction control thrusters are at the sides
  of the moonship with the bell of the ascent rocket engine underneath. The hatch
  allowing access to the lunar surface is seen at the front, with a round radar
  antenna at the top. Mission commander Gene Cernan is clearly visible through
  the triangular window. This spaceship performed gracefully, landing on the Moon
  and returning the Apollo astronauts to the orbiting command module in December
  of 1972. So where is Challenger now? Its descent stage remains at the Apollo
  17 landing site in the Taurus-Littrow valley. The ascent stage pictured was
  intentionally crashed nearby after being jettisoned from the command module
  prior to the astronauts' return to planet Earth.
4
5 Additional Search Terms:
6 • 'Apollo 17'
7 • 'Challenger lunar module'
8 • 'Reaction control thrusters'
9 • 'Ascent rocket engine'
10 • 'Lunar orbit'
11 • 'Lunar surface'
12 • 'Taurus-Littrow valley'
13
14 Searching Wikipedia for 'Apollo 17'...
15 Summary of 'Apollo 17': Apollo 1, initially designated AS-204, was planned to be
  the first manned mission of the Apollo program, the American undertaking to
  land the first man on the Moon. It was planned to launch on February 21, 1967,
  as the first low Earth orbital test of the Apollo command and service module.
  The mission never flew; a cabin fire during a launch rehearsal test at Cape
  Kennedy Air Force Station Launch Complex 34 on January 27 killed all three crew
  members—Command Pilot Gus Grissom, Senior Pilot Ed White, and Pilot Roger B.
  Chaffee—and destroyed the command module (CM). The name Apollo 1, chosen by the
  crew, was made official by NASA in their honor after the fire.
16 Immediately after the fire, NASA convened an Accident Review Board to determine
  the cause of the fire, and both chambers of the United States Congress
  conducted their own committee inquiries to oversee NASA's investigation. The
  ignition source of the fire was determined to be electrical, and the fire
  spread rapidly due to combustible nylon material and the high-pressure pure
  oxygen cabin atmosphere. Rescue was prevented by the plug door hatch, which
  could not be opened against the internal pressure of the cabin. Because the
  rocket was unfueled, the test had not been considered hazardous, and emergency
  preparedness for it was poor.
17 During the Congressional investigation, Senator Walter Mondale publicly
  revealed a NASA internal document citing problems with prime Apollo contractor
  North American Aviation, which became known as the Phillips Report. This
  disclosure embarrassed NASA Administrator James E. Webb, who was unaware of the
  document's existence, and attracted controversy to the Apollo program. Despite
  congressional displeasure at NASA's lack of openness, both congressional
  committees ruled that the issues raised in the report had no bearing on the
  accident.
18 Crewed Apollo flights were suspended for twenty months while the command
  module's hazards were addressed. However, the development and uncrewed testing
  of the lunar module (LM) and Saturn V rocket continued. The Saturn IB launch
  vehicle for Apollo 1, AS-204, was used for the first LM test flight, Apollo 5.
  The first successful crewed Apollo mission was flown by Apollo 1's backup crew
  on Apollo 7 in October 1968.
19
20 Searching Wikipedia for 'Challenger lunar module'...
21 Summary of 'Lunar module': The Apollo Lunar Module (LM), originally designated
  the Lunar Excursion Module (LEM), was the lunar lander spacecraft that was
  flown between lunar orbit and the Moon's surface during the United States'
  Apollo program. It was the first crewed spacecraft to operate exclusively in
  the airless vacuum of space, and remains the only crewed vehicle to land
  anywhere beyond Earth.
```

22 Structurally and aerodynamically incapable of flight through Earth's atmosphere, the two-stage Lunar Module was ferried to lunar orbit attached to the Apollo command and service module (CSM), about twice its mass. Its crew of two flew the Lunar Module from lunar orbit to the Moon's surface. During takeoff, the spent descent stage was used as a launch pad for the ascent stage which then flew back to the command module, after which it was also discarded.

23 Overseen by Grumman, the LM's development was plagued with problems that delayed its first uncrewed flight by about ten months and its first crewed flight by about three months. Regardless, the LM became the most reliable component of the Apollo-Saturn space vehicle. The total cost of the LM for development and the units produced was \$21.65 billion in 2016 dollars, adjusting from a nominal total of \$2.29 billion using the NASA New Start Inflation Indices.

24 Ten Lunar Modules were launched into space. Of these, six were landed by humans on the Moon from 1969 to 1972. The first two flown were tests in low Earth orbit: Apollo 5, without a crew; and Apollo 9 with a crew. A third test flight in low lunar orbit was Apollo 10, a dress rehearsal for the first landing, conducted on Apollo 11. The Apollo 13 Lunar Module functioned as a lifeboat to provide life support and propulsion to keep the crew alive for the trip home, when their CSM was disabled by an oxygen tank explosion en route to the Moon.

25 The six landed descent stages remain at their landing sites; their corresponding ascent stages crashed into the Moon following use. One ascent stage (Apollo 10's Snoopy) was discarded in a heliocentric orbit after its descent stage was discarded in lunar orbit. The other three LMs were destroyed during controlled re-entry in the Earth's atmosphere: the four stages of Apollo 5 and Apollo 9 each re-entered separately, while Apollo 13's Aquarius re-entered as a unit.

26

27 Searching Wikipedia for 'Reaction control thrusters'...

28 Summary of 'Reaction control system': A reaction control system (RCS) is a spacecraft system that uses thrusters to provide attitude control and translation. Alternatively, reaction wheels can be used for attitude control, rather than RCS. Use of diverted engine thrust to provide stable attitude control of a short-or-vertical takeoff and landing aircraft below conventional winged flight speeds, such as with the Harrier "jump jet", may also be referred to as a reaction control system.

29 Reaction control systems are capable of providing small amounts of thrust in any desired direction or combination of directions. An RCS is also capable of providing torque to allow control of rotation (roll, pitch, and yaw).

30 Reaction control systems often use combinations of large and small (vernier) thrusters, to allow different levels of response.

31 Searching Wikipedia for 'Ascent rocket engine'...

32 Summary of 'Ascent propulsion system': The ascent propulsion system (APS) or lunar module ascent engine (LMAE) is a fixed-thrust hypergolic rocket engine developed by Bell Aerosystems for use in the Apollo Lunar Module ascent stage. It used Aerozine 50 fuel, and N2O4 oxidizer. Rocketdyne provided the injector system, at the request of NASA, when Bell could not solve combustion instability problems.

33

34 Searching Wikipedia for 'Lunar orbit'...

35 Summary of 'Lunar orbit': In astronomy and spaceflight, a lunar orbit (also known as a selenocentric orbit) is an orbit by an object around Earth's Moon. In general these orbits are not circular. When farthest from the Moon (at apoapsis) a spacecraft is said to be at apolune, apocynthion, or aposelene. When closest to the Moon (at periapsis) it is said to be at perilune, pericynthion, or periselene. These derive from names or epithets of the moon goddess.

36 Lunar orbit insertion (LOI) is an orbit insertion maneuver used to achieve lunar orbit.

37 Low lunar orbit (LLO) is an orbit below 100 km (62 mi) altitude. These have a period of about 2 hours. They are of particular interest in the exploration of the Moon, but suffer from gravitational perturbations that make most unstable, and leave only a few orbital trajectories possible for indefinite frozen orbits. These would be useful for long-term stays in LLO.

38

38 orbits. These would be useful for long-term stays in LEO.

39 Searching Wikipedia for 'Lunar surface'...

40 Summary of 'Moon landing': A Moon landing or lunar landing is the arrival of a spacecraft on the surface of the Moon, including both crewed and robotic missions. The first human-made object to touch the Moon was Luna 2 in 1959.

41 In 1969 Apollo 11 was the first crewed mission to land on the Moon. There were six crewed landings between 1969 and 1972, and numerous uncrewed landings. All crewed missions to the Moon were conducted by the Apollo program, with the last departing the lunar surface in December 1972. After Luna 24 in 1976 there were no soft landings on the Moon until Chang'e 3 in 2013. All soft landings took place on the near side of the Moon until January 2019, when Chang'e 4 made the first landing on the far side of the Moon.

42 Searching Wikipedia for 'Taurus-Littrow valley'...

43 Summary of 'Taurus-Littrow': Taurus-Littrow is a lunar valley located on the near side at the coordinates 20.0°N 31.0°E / 20.0; 31.0. It served as the landing site for the American Apollo 17 mission in December 1972, the last crewed mission to the Moon. The valley is located on the southeastern edge of Mare Serenitatis along a ring of mountains formed between 3.8 and 3.9 billion years ago when a large object impacted the Moon, forming the Serenitatis basin and pushing rock outward and upward.

45 Taurus-Littrow is located in the Taurus mountain range and south of Littrow crater, features from which the valley received its name. The valley's name, coined by the Apollo 17 crew, was approved by the International Astronomical Union in 1973.

46 Data collected during Apollo 17 indicate that the valley is composed primarily of feldspar-rich breccia in the large massifs surrounding the valley and basalt underlying the valley floor, covered by an unconsolidated layer of mixed material formed by various geologic events. Taurus-Littrow was selected as the Apollo 17 landing site with the objectives of sampling highland material and young volcanic material at the same location.

47

47

48 RAG Generated Description:

49

50 This photograph showcases a historic moment in space exploration, taken during the Apollo 17 mission in December 1972. The image captures the lunar module Eagle, piloted by Eugene Cernan and Harrison Schmitt, as it descends onto the surface of the Moon's south pole.

51

52 In the foreground, the landscape stretches out into the vast expanse of Taurus-Littrow valley, a region located on the near side of the Moon at coordinates 20.0°N 31.0°E. The valley was chosen by the Apollo 17 crew as their landing site due to its unique geological features and scientific significance.

53

54 The lunar module's descent stage can be seen in the distance, while the ascent engine, a fixed-thrust rocket engine developed by Bell Aerosystems, is visible on the side of the module. This engine was used to propel the spacecraft back into lunar orbit after the landing.

55

56 Reaction control thrusters, small engines that provide attitude control and translation, are also visible on the exterior of the lunar module. These systems played a crucial role in stabilizing the spacecraft during its descent and ascent.

57

58 The reaction control thrusters are paired with the ascent rocket engine, which is a hypergolic rocket engine that uses Aerozine 50 fuel and N2O4 oxidizer to produce thrust. This engine was used to propel the lunar module away from the Moon's surface and back into orbit.

59

60 As the spacecraft takes off, it can be seen leaving behind a cloud of debris, including regolith, the fine dust that covers the Moon's surface. The lunar module is also accompanied by two astronauts, Cernan and Schmitt, who are suited up in their spacesuits to protect themselves from the harsh conditions on the Moon.

61

62 The image also includes several notable features of the Taurus-Littrow valley, including the Littrow crater, which can be seen in the distance. The valley is composed primarily of feldspar-rich breccia, a type of rock that forms when fragments of other rocks are welded together by heat and pressure.

63

64 Overall, this photograph captures an iconic moment in space exploration history, showcasing the technological achievements of the Apollo 17 mission and providing a glimpse into the fascinating geology and landscape of the Moon's surface.

65

66 Link to this photo:

67 <https://apod.nasa.gov/apod/image/2412/AS17-149-22859-2v2SmlWmk.jpg>

68

Deliverables

readme.txt

So-called “read me” files are a common way for developers to leave high-level notes about their applications. Here’s an example of a [README file](#) for the Apache Spark project. They usually contain details about required software versions, installation instructions, contact information, etc. For our purposes, your readme.txt file will be a way for you to describe the approach you took to complete the assignment so that, in the event you may not quite get your solution working correctly, we can still award credit based on what you were trying to do. Think of it as the verbalization of what your code does (or is supposed to do). Your readme.txt file should contain the following:

1. **Name:** Your name and JHED ID
2. **Module Info:** The Module name/number along with the title of the assignment and its due date
3. **Approach:** a detailed description of the approach you implemented to solving the assignment. Be as specific as possible. If you are sorting a list of 2D points in a plane, describe the class you used to represent a point, the data structures you used to store them, and the algorithm you used to sort them, for example. The more descriptive you are, the more credit we can award in the event your solution doesn’t fully work.
4. **Known Bugs:** describe the areas, if any, where your code has any known bugs. If you’re asked to write a function to do a computation but you know your function returns an incorrect result, this should be noted here. Please also state how you would go about fixing the bug. If your code produces results correctly you do not have to include this section.

Please submit your **main.py**, **rag.py**, **apod.py**, and **wiki.py** source code files along with a Word/PDF file containing screenshots of your output in a file called JHEDID_mod9.pdf (ex: jkovba1_mod9.pdf). Please do not ZIP your files together.

Recap:

1. readme.txt
2. main.py, rag.py, apod.py, wiki.py, api.py
3. results.txt (output generated from running main.py)
4. nasa_apod_image.jpg
5. A screenshot(s) of your outputs from main.py:
 - a. Output to terminal (from running main.py)
 - b. Contents of results.txt (multiple screenshots for this are fine).
 - c. Photo of the day image

*Note: Since you are using an LLM, the output results will differ slightly. Make sure to adhere to the format of the information shown in the example output. **No reformatting of material returned from the LLM is necessary.***

Please let us know if you have any questions via Teams or email!

Grading

We wanted to provide a few notes on grading to help you be as successful as possible on the assignments:

1. Following instructions closely is an important part of good software development practices. Even small details like a file in the wrong format can negatively impact software interoperability. For this reason, **we ask that you please follow the instructions as closely as possible. This includes matching the output format of the Sample Solutions exactly, submitting the required files in the correct formats, etc.**
2. The Readings + Video Lectures provide 90%+ of the skills necessary to complete this Assignment. While we encourage experimentation, we ask that you focus on the use of what we learned in this Module, and previous Modules, to develop your solution. **If you'd like to try other techniques, we ask that you please practice them separately and do not submit them as your assignment solution.**
3. Because the Readings + Video Lectures constitute a large majority of skills required to complete this assignment, we ask that you use external references as judiciously as possible. That is, **you may refer to external resources as needed, but such resources should be (informally) cited in your README and code borrowed from these resources must not be copy/pasted.**
4. **Unless otherwise specified, you do not need to worry about any error-checking or input sanitization.** We'll learn techniques for detecting and correcting errors later in the course and explicitly state when it's required.
5. **The output of your solution should match that of the Sample Outputs exactly.** It is important not just to have a functional solution but also one that adheres to the exact formatting specifications. For example, if another piece of code was expecting your results in the specified format but you added an extra word, or even an extra space, it could potentially break that code. Producing the proper output is also a good measure of your understanding in how to manipulate and display information that your code produces.

Note: the format of the output is different than how the formatted output is displayed on the screen. Depending on your screen size, the width of your IDE window, etc., the positioning/wrapping of your output text may appear slightly differently, though still in the same format, as ours.